

# Architecting Relational Databases

A structural guide to data models, normalization, and SQL infrastructure.

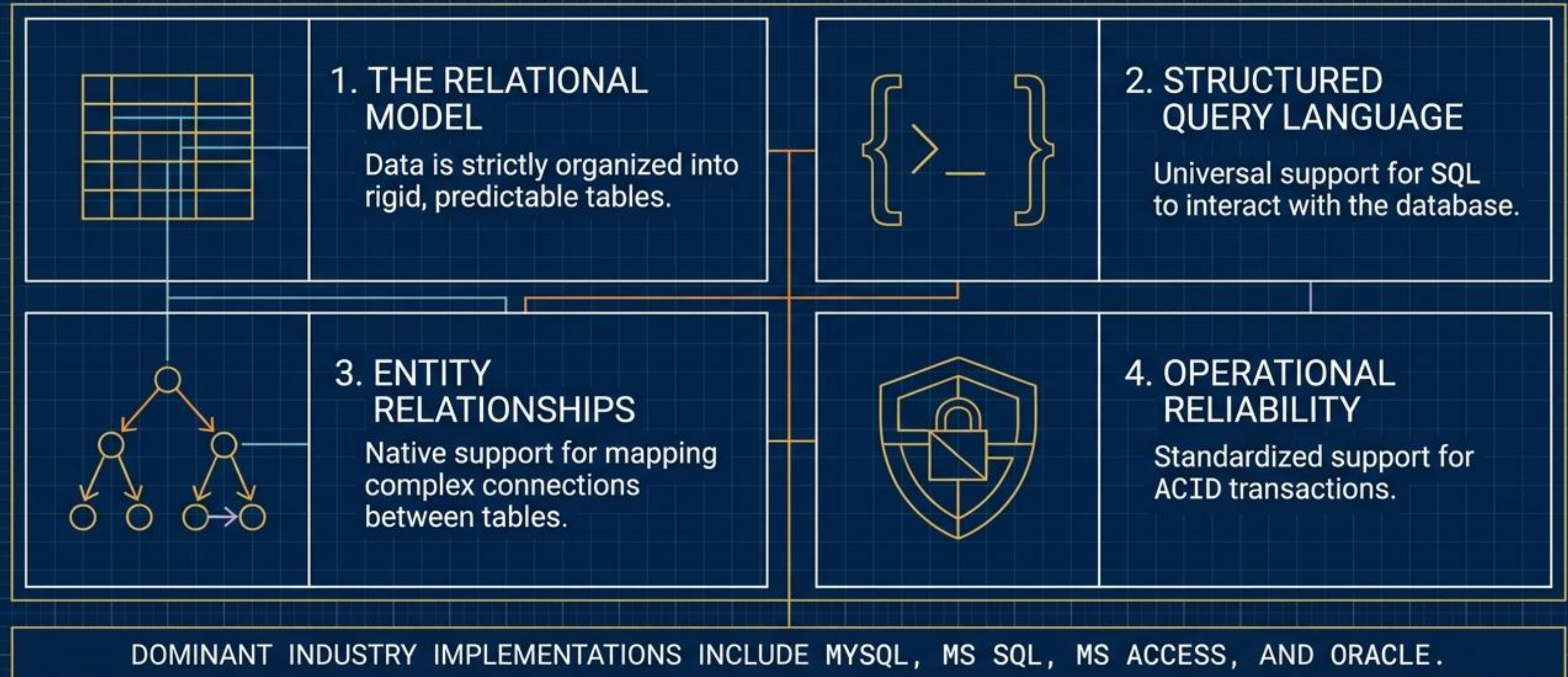
COM 769 | Scalable Advanced Software  
Systems | Ulster University





# THE PARADIGM THAT DEFINES STRUCTURED DATA STORAGE

Proposed in the 1970s by E.F. Codd, Relational Database Management Systems (RDBMS) introduced a strict, mathematical approach to data.





# THE STRUCTURAL ANATOMY OF A RELATION

Relational models map data into rigid tables, generally representing a single entity type (e.g., a customer or student record).

## Attribute / Field

A single vertical column defining one specific property of the entity

## Tuple / Record

A single horizontal row containing one complete entity entry

| Customer ID | Tax ID      | Name       | Address          | [More fields...] |
|-------------|-------------|------------|------------------|------------------|
| 1234567890  | 555-5512222 | Munmun     | 323 Broadway     | ...              |
| 2223344556  | 555-5523232 | Wile E.    | 1200 Main Street | ...              |
| 3334445563  | 555-5533323 | Ekta       | 871 1st Street   | ...              |
| 4232342432  | 555-5325523 | E. F. Codd | 123 It Way       | ...              |

## Relation / Base Relvar

The entire table structure itself



# Defining unique identifiers through database keys

## Primary Key (PK)

**Function:** Offers a completely unique identifier for a single tuple.

**Format:** Frequently an auto-incrementing numeric value.

Primary Key

| Store ID | Purchase Location |
|----------|-------------------|
| 1        | Los Angeles       |
| 2        | New York          |
| 3        | San Francisco     |

## Composite Key (CK)

**Function:** Identifies a unique tuple using a combination of two or more columns.

**Format:** May employ auto-incrementing on one, some, or all involved columns.

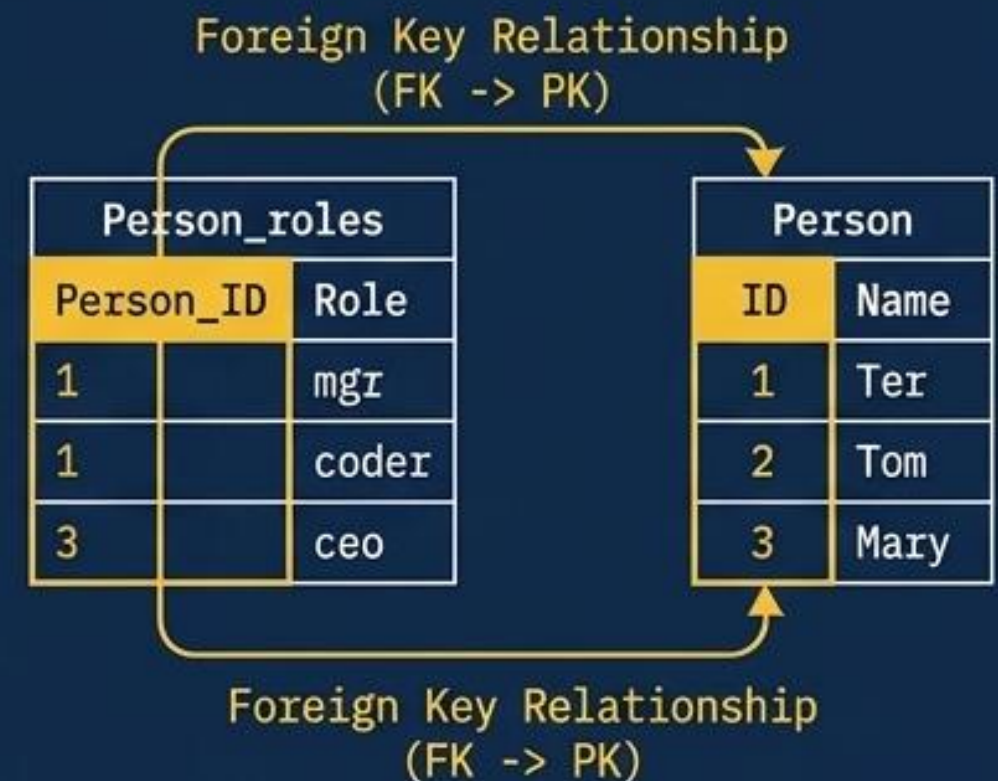
Composite Key

| Customer ID | Store ID | Purchase Location |
|-------------|----------|-------------------|
| 1           | 1        | Los Angeles       |
| 1           | 3        | San Francisco     |
| 2           | 1        | Los Angeles       |
| 3           | 2        | New York          |
| 4           | 3        | San Francisco     |

## Foreign Key (FK)

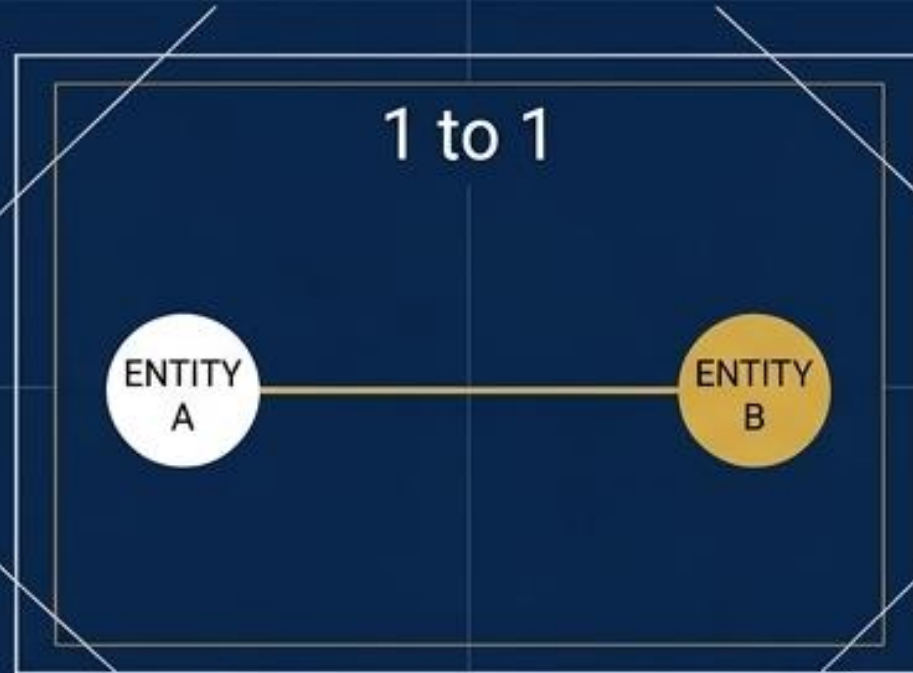
**Function:** A field in one table that uniquely identifies a key in a foreign table.

**Format:** Facilitates selection across tables; enforces referential integrity.

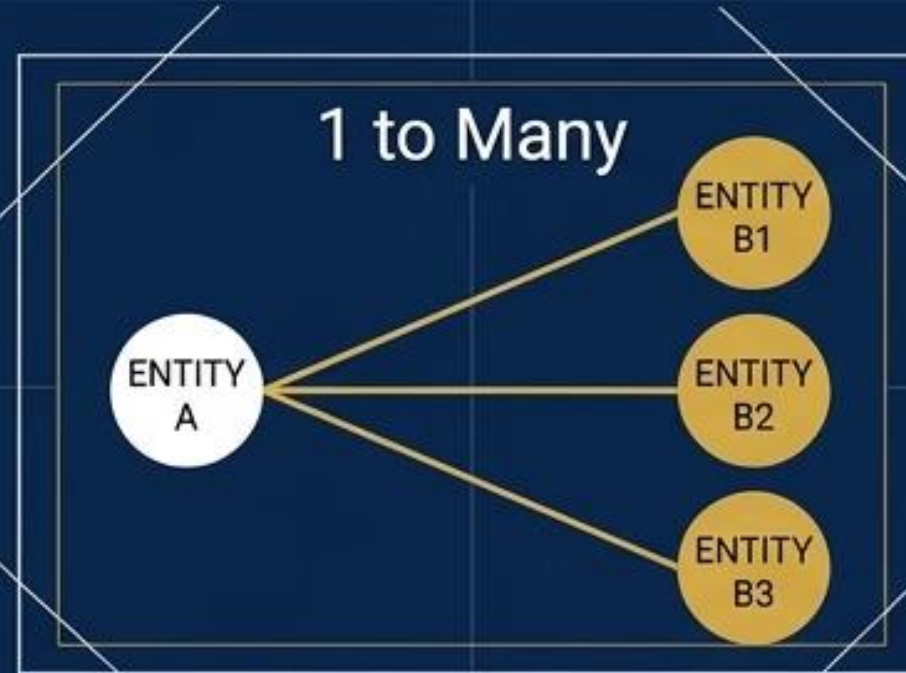




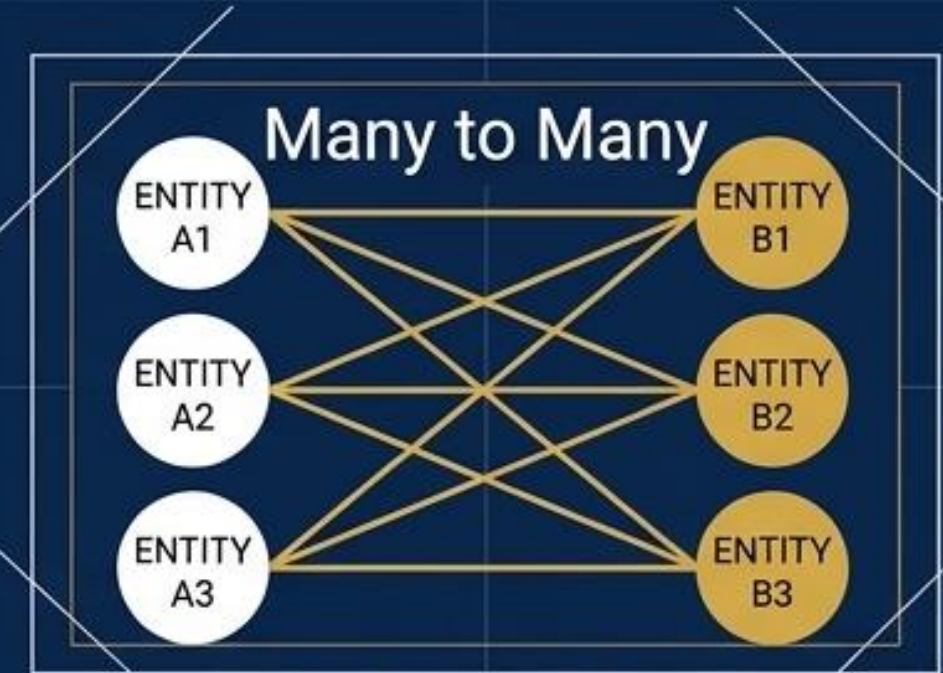
# Enforcing rules and mapping complex entity relationships



e.g., One person has exactly one social security number.



e.g., One person holds multiple bank accounts.



e.g., Books have multiple authors, and authors write multiple books.

## The Rule of Referential Integrity

A Foreign Key may ONLY contain values from a specified column in the target table (or a null value). This provides strict mathematical assurance that related information actually exists.

 **Deletion protocols:** When deleting a record containing an FK, the RDBMS must be configured to either cascade the deletion to related tables or actively prevent the deletion to protect structural integrity.



# Minimizing redundancy through Normalization (1NF)

Normalization reorganizes attributes to minimize redundancy, increasing storage efficiency. A database is only considered normalized once it reaches 3NF.

## First Normal Form (1NF) Building Codes

Eliminate repetition in individual tables.

Create a separate table for each related data/entity type.

Identify each record with a primary key.

**CRITICAL:** Contain only atomic values.

## Violation: Non-Atomic Value

| Customer ID | First Name | Surname   | Telephone Number              |
|-------------|------------|-----------|-------------------------------|
| 123         | Robert     | Ingram    | 555-861-2025                  |
| 456         | Jane       | Wright    | 555-403-1659,<br>555-776-4100 |
| 789         | Maria      | Fernandez | 555-808-9633                  |



## Resolution: 1NF Satisfied

| Customer ID | First Name | Surname   | Telephone Number |
|-------------|------------|-----------|------------------|
| 123         | Robert     | Ingram    | 555-861-2025     |
| 456         | Jane       | Wright    | 555-403-1659     |
| 456         | Jane       | Wright    | 555-776-4100     |
| 789         | Maria      | Fernandez | 555-808-9633     |



# Achieving Second Normal Form (2NF)

## The 2NF Rule

To reach 2NF, a table must first satisfy 1NF, and all non-key attributes must be FULLY functionally dependent on the primary key.

## The Violation: Partial Dependency

| Customer ID | Store ID | Purchase Location |
|-------------|----------|-------------------|
| 1           | 1        | Los Angeles       |
| 1           | 3        | San Francisco     |
| 2           | 1        | Los Angeles       |
| 3           | 2        | New York          |
| 4           | 3        | San Francisco     |

Partial Dependency

The Purchase Location column only depends on Store ID, which is just a partial element of the primary composite key.

## The Solution: Table Splitting

| Customer ID | Store ID |
|-------------|----------|
| 1           | 1        |
| 1           | 3        |
| 2           | 1        |
| 3           | 2        |
| 4           | 3        |

| Store ID | Purchase Location |
|----------|-------------------|
| 1        | Los Angeles       |
| 2        | New York          |
| 3        | San Francisco     |

The table is split. Store location data is abstracted into its own distinct table, satisfying 2NF.



# The final baseline: Third Normal Form (3NF)

## Axiom 1:

"Every non-prime attribute of R is non-transitively dependent on every key of R."

– E.F. Codd (The Formal Definition)

## Axiom 2:

"Every non-key attribute must provide a fact about the key, the whole key, and nothing but the key."

– Bill Kent (The Practical Definition)

## The Violation: Transitive Dependency



| Book ID | Genre ID | Genre Type | Price |
|---------|----------|------------|-------|
| 1       | 1        | Gardening  | 25.99 |
| 2       | 2        | Sports     | 14.99 |
| 3       | 1        | Gardening  | 10.00 |
| 4       | 3        | Travel     | 12.99 |
| 5       | 2        | Sports     | 17.99 |

A transitive dependency loop exists between Genre ID and Genre Type.

## The Solution: 3NF Achieved

Table 1: Books

| Book ID | Genre ID | Price |
|---------|----------|-------|
| 1       | 1        | 25.99 |
| 2       | 2        | 14.99 |
| 3       | 1        | 10.00 |
| 4       | 3        | 12.99 |
| 5       | 2        | 17.99 |

Table 2: Genres

| Genre ID | Genre Type |
|----------|------------|
| 1        | Gardening  |
| 2        | Sports     |
| 3        | Travel     |

Splitting into two tables isolates the Genre mapping, successfully achieving a normalized 3NF database.



# The operational guarantee of ACID transactions

[A]

## Atomicity

An entire operation is performed entirely, or nothing is performed at all. No partial states.

[C]

## Consistency

Any transaction will successfully bring the database from one mathematically valid state to another valid state.

[I]

## Isolation

Concurrent transactions occur invisibly to each other; they will not interfere.

[D]

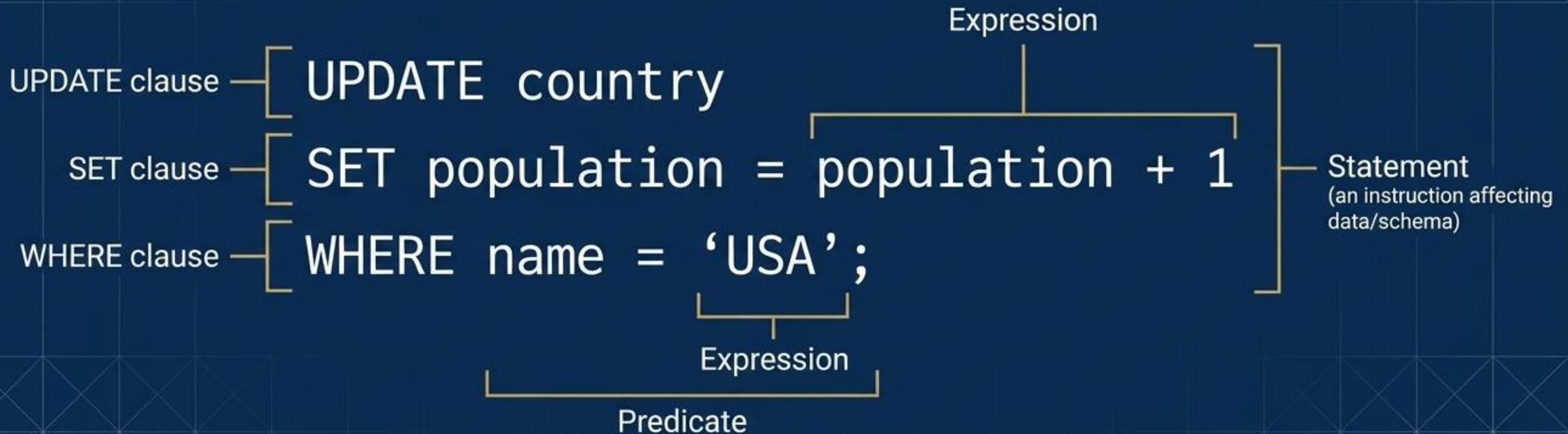
## Durability

Once a transaction has been formally committed, it remains permanently stored, surviving power loss or system errors.



# Deconstructing Structured Query Language (SQL)

SQL is the primary mechanism for interacting with an RDBMS, subdivided into strict logical language elements.





# The SQL operational toolkit

| Data Manipulation (CRUD) |                                           | Data Structuring & Filtering |                                                                    |
|--------------------------|-------------------------------------------|------------------------------|--------------------------------------------------------------------|
| SELECT                   | Retrieve data from a database (Querying). | WHERE                        | Restricts the operation of a statement based on predicates.        |
| INSERT                   | Push new data into a database.            | LIMIT                        | Caps the absolute number of records returned in a result set.      |
| UPDATE                   | Modify existing records.                  | JOIN                         | Combines rows across 2+ tables based on related columns.           |
| DELETE                   | Permanently remove records.               | DISTINCT                     | Strips out duplicates from a result set.                           |
|                          |                                           | ORDER BY                     | Sequentially sorts the result set by specified columns.            |
|                          |                                           | GROUP BY                     | Collects data across multiple records to group aggregated results. |



# Data Type selection and memory allocation

| String/Text Types                                                                                                                                                                                                                                                   | Binary Types                                                                                                                           | Numeric Types                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• <b>CHAR</b>: Fixed length, up to 255 characters.</li><li>• <b>VARCHAR</b>: Variable length, up to 65,535 characters.</li><li>• <b>TEXT</b>: Large strings (cannot be ordered or typically used in WHERE clauses).</li></ul> | <ul style="list-style-type: none"><li>• <b>BINARY</b>: Standard binary values.</li><li>• <b>BLOB</b>: Binary string objects.</li></ul> | <ul style="list-style-type: none"><li>• <b>INT</b>: 4 bytes (Signed: -2.14B to 2.14B   Unsigned: 0 to 4.29B).</li><li>• <b>BIGINT</b>: 8 bytes (Massive scale up to 18.4 Quintillion).</li></ul> |

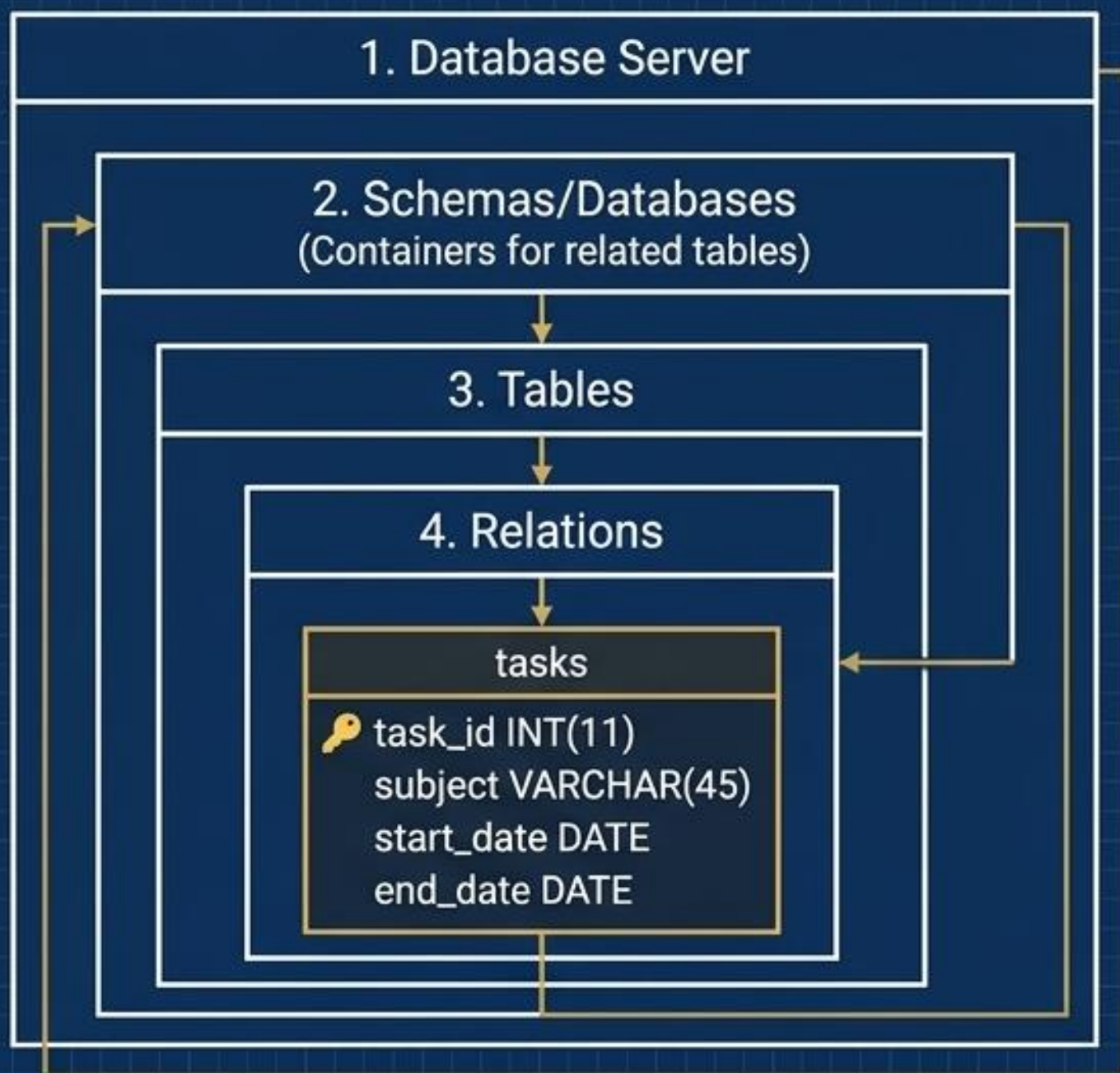


Data type limits vary by RDBMS implementation. Correct specification is mathematically imperative—allocating **BIGINT** when **INT** suffices creates cascading performance and storage degradation.



# The RDBMS storage hierarchy

## The Hierarchy Map



## Executing the Schema

### Block 1 (Initialization):

```
CREATE SCHEMA example_schema ;
```

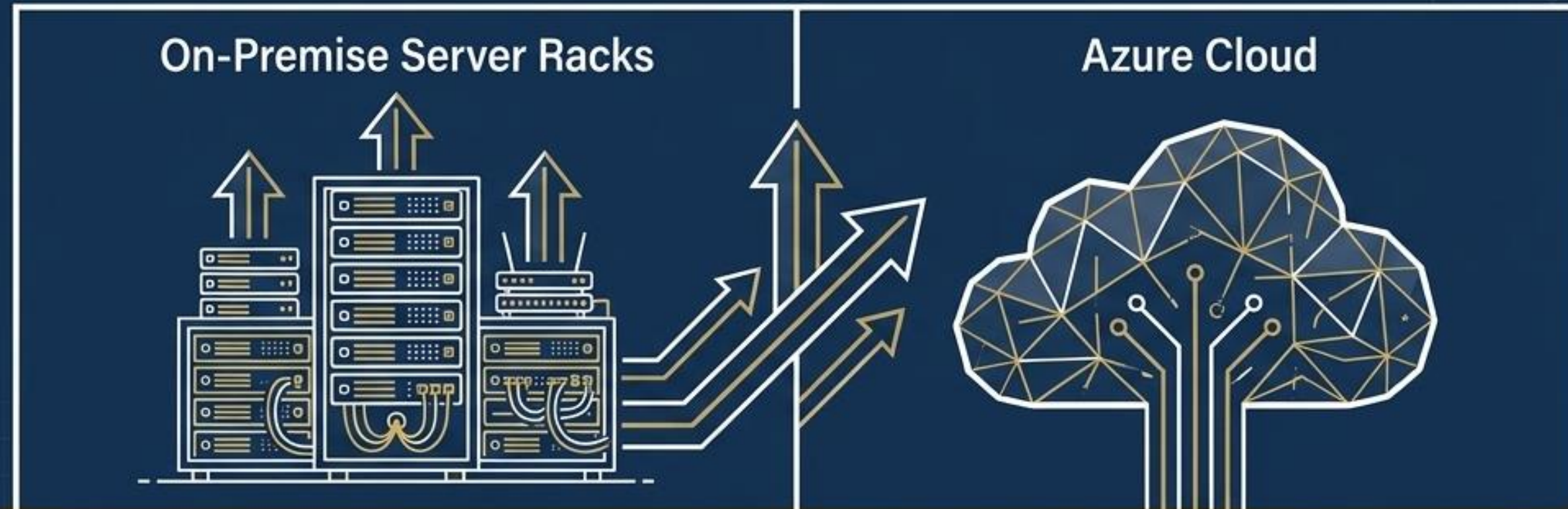
### Block 2 (Building the Table):

```
CREATE TABLE IF NOT EXISTS example_schema.tasks (  
  task_id INT(11) NOT NULL AUTO_INCREMENT,  
  subject VARCHAR(45) DEFAULT NULL,  
  PRIMARY KEY (task_id)  
) ENGINE=InnoDB;
```



# Cloud deployment infrastructure: Azure SQL

Within the Azure SQL platform umbrella, deployment choices dictate architectural freedom versus operational overhead.



## The “Managed Instance” Strategy

- Ideal for migrating legacy systems to the cloud **WITHOUT** rearchitecting applications.
- **Instance-Scoped vs. Database-Scoped**: Managed instances unlock features tied to the entire SQL Server instance (like Machine Learning Services), rather than just features isolated to a single database.
- **The Cloud Trade-off**: You gain access to these instance-wide features while Azure abstracts away the underlying OS and infrastructure maintenance.



# The Architectural Boundary: When NOT to use an RDBMS

Relational databases excel at reliable storage of transactional records (like financial logs) and address books. However, their rigid structure makes them the wrong choice for specific workloads.

## Diagnostic Exclusion Criteria



### Unstructured Volume

High-volume log data that does not actually require strict ACID consistency checks.



### Heavy Objects

Massive storage of binary objects (images, videos, files).



### Nested Documents

Storage of deeply nested XML or JSON documents.



### Fluid Architecture

Any system requiring a dynamically changing or highly variable schema definition over time.